Detailed BTP Progress Report

Resilient Swarm Navigation via Trust-Aware Multi-Agent Reinforcement Learning (TA-MAPPO)

Submitted By

Punarvitha Kommana	Roll No: S20230020361
Srinivasa Rao	Roll No: S20230030386
Varthala Khyathi Yadav	Roll No: S20230030418

Under the Guidance of

Dr. Sakthi Swarrup J

Department of Mathematics and Data Science

IIIT Sri City

2026

1 Abstract

Autonomous drone swarms are increasingly deployed in surveillance, search-and-rescue, logistics, and reconnaissance applications, where reliable decentralized coordination is essential. In such systems, multi-agent reinforcement learning offers a promising framework for cooperative control; however, its effectiveness is limited when communication among agents is unreliable or adversarial. Deceptive or inconsistent inter-agent information can degrade coordination, increase collision risk, and destabilize swarm performance.

This report presents ongoing work on a Trust-Aware Multi-Agent Proximal Policy Optimization (TA-MAPPO) framework for resilient decentralized swarm navigation. The current implementation includes a continuous-control multi-agent simulation environment, LiDAR-based perception, PPO-based training, reward engineering, and preliminary evaluation under random-spawn and dense-cluster conditions. Experimental results from Phase A indicate stable swarm coordination, with a success rate of 99.68% in random-spawn scenarios and 95.78% in dense-cluster scenarios.

The trust-gating and adversarial-communication components, which form the core of the proposed TA-MAPPO architecture, are currently under development. The present work establishes the simulation and learning framework required for subsequent implementation and evaluation of trust-aware swarm resilience mechanisms.

2 Brief Project Overview

2.1 Problem Statement

In continuous-space multi-agent navigation tasks, drones must coordinate movement within a confined arena while avoiding obstacles and neighboring agents using only local sensing and limited neighbor communication. Although multi-agent reinforcement learning has demonstrated strong potential for decentralized coordination, many existing frameworks still rely on the assumption that inter-agent communication is reliable, consistent, and truthful.

This assumption becomes a critical weakness in adversarial environments. In this project, we consider a Byzantine threat model in which a variable subset of compromised drones intentionally transmits deceptive positional and velocity information. By broadcasting state estimates that are inconsistent with their actual ground-truth kinematics, such agents can influence the local observation vectors of nearby honest drones and alter the actor network's input before action selection.

In dense swarm settings, even small amounts of misleading information can propagate quickly through the policy. A false proximity broadcast may cause an honest agent to initiate an unnecessary evasive maneuver, which in turn can push it into the path of another agent and trigger a chain of collisions. Similarly, misleading motion or position data may create artificial congestion, slow down goal convergence, or disrupt the spatial coherence required for effective collective movement. Over time, this can lead to collision cascades, congestion deadlocks, and swarm fragmentation.

Conventional MARL methods are not designed to evaluate the trustworthiness of incoming communication before it affects policy inference. This creates a clear need for a trust-aware swarm navigation framework that can filter unreliable state estimates and preserve decentralized

coordination under adversarial conditions.

2.2 Objectives

The overarching objective of this project is to design, implement, and validate a resilient swarm intelligence framework capable of operating under adversarial communication conditions. The specific objectives are:

1. To develop a continuous-space multi-agent swarm simulation environment supporting local LiDAR sensing and decentralized navigation.
2. To establish a baseline Vanilla MAPPO policy for multi-agent swarm coordination.
3. To model Byzantine adversarial behavior by injecting deceptive kinematic broadcasts from compromised agents.
4. To design a proactive trust evaluation module that compares neighbor broadcasts with trusted local sensor observations.
5. To integrate the trust module into a Trust-Aware Multi-Agent Proximal Policy Optimization (TA-MAPPO) framework that filters unreliable communication before policy inference.
6. To benchmark the proposed TA-MAPPO framework against the baseline under dense-cluster and adversarial scenarios using success rate, collision rate, and convergence stability.

2.3 Research Motivation

Drone swarms are increasingly deployed in environments where communication integrity cannot always be guaranteed. Existing reinforcement learning frameworks prioritize coordination efficiency but often ignore adversarial robustness.

Biological immune systems provide a natural model for anomaly detection and selective trust regulation. T-cell immune mechanisms continuously evaluate signals and suppress harmful interactions before they affect the overall system.

Inspired by these principles, this project integrates bio-inspired trust filtering within a swarm reinforcement learning framework. The research combines:

- Bio-inspired anomaly detection
- Trust-aware communication
- Decentralized swarm intelligence
- Multi-Agent Reinforcement Learning

No existing framework currently integrates trust gating, CTDE learning, continuous-control swarm navigation, and adversarial resilience within a unified drone swarm environment.

2.4 Expected Contribution

The expected contributions of this project include:

2.4.1 TA-MAPPO Framework

A trust-aware extension of MAPPO integrating bio-inspired trust evaluation into decentralized swarm reinforcement learning.

2.4.2 Trust-Gated Communication

A proactive trust-filtering system that suppresses deceptive communication before policy inference instead of penalizing malicious agents after failure occurs.

2.4.3 Social-Distancing Reward

A novel reward-engineering mechanism designed to reduce collision cascades and improve dense swarm coordination.

2.4.4 Reproducible Swarm Environment

A PettingZoo-based continuous multi-agent simulator supporting:

- Obstacle navigation
- Dense-cluster testing
- Adversarial communication
- Statistical reproducibility

3 Methodology and System Design

3.1 Proposed Approach and Research Phases

The proposed methodology is divided into four incremental phases to systematically develop, test, and extend the swarm navigation framework. This phased design ensures that each stage builds on the previous one, starting from a stable baseline and gradually introducing navigation complexity, adversarial behavior, and trust-aware defense mechanisms.

3.1.1 Phase A: Baseline Swarm Coordination

The first phase focuses on establishing a reliable decentralized swarm navigation environment. In this phase, the system is implemented as a continuous-control multi-agent simulation with 10 drones operating in a 20×20 arena. Each drone is equipped with local LiDAR sensing and can exchange kinematic information with nearby agents. A baseline Vanilla MAPPO policy is trained under the assumption that all communication is honest and all agents are cooperative.

The purpose of this phase is to verify that the environment, observation structure, reward design, and PPO training pipeline are functioning correctly. This phase provides the foundation for all later experiments because it establishes whether the swarm can coordinate effectively before any trust or adversarial components are introduced. The key outputs of this phase are stable navigation, basic collision avoidance, and measurable success in random-spawn scenarios.

3.1.2 Phase B: Complex Navigation and Synchronization

The second phase extends the baseline environment by introducing obstacle-rich maps and denser swarm configurations. This phase is aimed at testing whether the learned policy can maintain coordination when the environment becomes more constrained. The swarm must now deal with tighter corridors, limited free space, and higher probabilities of local congestion.

During this phase, the reward structure is refined to encourage cohesive motion, safe spacing, and goal-directed progress while avoiding collisions and deadlocks. This stage is important because adversarial communication should not be added before the policy is already stable under difficult but non-adversarial conditions. In other words, the swarm must first learn to handle environmental stress before being tested under communication stress.

3.1.3 Phase C: Adversarial Modeling and Trust Evaluation

The third phase introduces the Byzantine threat model into the simulation. Here, a subset of drones is treated as compromised and is allowed to transmit deceptive kinematic broadcasts such as false position or velocity information. These misleading messages simulate real-world communication corruption or malicious interference in a decentralized swarm.

At the same time, a trust evaluation module is developed. This module compares received neighbor broadcasts with trusted local LiDAR observations and computes a trust score for each communication source. If the broadcast is inconsistent with sensor evidence, its influence is reduced before it reaches the policy network. The main objective of this phase is not yet full defense, but rather the development and validation of a reliable trust-estimation mechanism that can distinguish between honest and suspicious communication.

3.1.4 Phase D: Full TA-MAPPO Integration and Defense

The final phase integrates the trust module into the reinforcement learning pipeline to form the complete TA-MAPPO framework. In this stage, unreliable communication is filtered or attenuated before policy inference, so the actor network makes decisions using both local sensing and selectively trusted neighborhood information. This creates a trust-aware policy that is expected to be more resilient to adversarial broadcasts than the vanilla MAPPO baseline.

The completed framework is then evaluated across multiple scenarios, including random-spawn settings, dense clusters, obstacle maps, and adversarial communication conditions. Performance is measured using success rate, collision rate, time to goal, and convergence stability. The objective of this phase is to quantify how much the trust-aware design improves swarm robustness under realistic and malicious operating conditions.

3.1.5 System Flow

The overall workflow of the proposed method can be summarized as follows:

- Initialize the swarm environment with drones, obstacles, sensing, and communication channels.
- Train the baseline MARL policy under honest communication.
- Test the baseline in increasingly complex navigation scenarios.
- Inject deceptive broadcasts from compromised agents.
- Compute trust scores using local sensing and communication consistency.
- Filter low-trust features before policy inference.
- Execute actions using the TA-MAPPO actor-critic framework.
- Compare the final system against the baseline using standard performance metrics.

3.2 Environment Design and MDP Formulation

The decentralized drone navigation problem is modeled as a cooperative, partially observable Markov Decision Process (Dec-POMDP), defined by the tuple $\langle \mathcal{N}, \mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma \rangle$, where $\mathcal{N}$ is the set of agents, $\mathcal{S}$ the global state space, $\mathcal{O}$ the local observation function, $\mathcal{A}$ the action space, $\mathcal{T}$ the transition dynamics, $\mathcal{R}$ the reward function, and γ the discount factor. The simulation is implemented using the PettingZoo Parallel API, which supports simultaneous, continuous-control interactions for $|\mathcal{N}| = 10$ agents operating in a bounded 20×20 unit two-dimensional arena.

3.2.1 Observation Space

Since each agent executes its policy in a fully decentralized manner, the global state $\mathcal{S}$ is not directly accessible to any individual drone during inference. Instead, each agent i constructs a local observation vector $o_i \in \mathcal{O}$ from three modular components: self-kinematics, local LiDAR sensing, and neighbor communication.

Self-Kinematics (6 features). The agent's own dynamic state is encoded as a 6-dimensional

vector comprising normalized velocity $(v_x/v_{\max}, v_y/v_{\max})$ where $v_{\max} = 2.0$ m/s; unit direction to goal $\hat{u}_g = (g-p_i)/|g-p_i|$, a 2D unit vector pointing from the agent's current position p_i toward the shared global goal g; normalized distance to goal $d_g/d_{\max}$, where $d_{\max} = 20\sqrt{2} \approx 28.28$ is the maximum possible Euclidean distance within the arena; and heading angle $\theta = \arctan 2(v_y, v_x)$, representing the current direction of motion.

LiDAR Sensing — Trusted Local Perception (16 features in Phase A). Each drone simulates a 2D LiDAR sensor to perceive its immediate physical surroundings. In Phase A, the sensor is configured with 16 equispaced rays covering a full 360° field of view, with an angular separation of 22.5° per ray. Each ray $k \in \{1, 2, \ldots, 16\}$ is cast outward from the drone's center position p_i along the direction vector

$$\hat{r}_k = [\cos(\alpha_k), \sin(\alpha_k)],$$

where

$$\alpha_k = \frac{2\pi k}{16}.$$

The ray-casting algorithm computes the minimum intersection distance d_k against three categories of physical boundaries. For arena walls, the parametric intersection distance t is computed along the ray direction for each of the four boundaries:

$$t_{\text{wall}} = \frac{b - p_{i,\text{axis}}}{\hat{r}_{k,\text{axis}}}$$

where b is the boundary coordinate (0 or 20) and axis $\in \{x, y\}$. For static obstacles in Phase B, the algorithm solves a ray-circle intersection using the standard quadratic formula, adding the drone's physical radius $r_{\text{drone}} = 0.15$ to each obstacle radius as a safety buffer. For neighboring drones, each active neighbor j is modeled as a circle of radius $r_{\text{drone}} = 0.15$; the algorithm projects the relative vector $(p_j - p_i)$ onto the ray direction and determines an intersection if the perpendicular distance from the ray to the neighbor's center is less than r_{drone}.

The final reading for ray k is:

$$d_k = \min(t_{\text{wall},k}, t_{\text{obs},k}, t_{\text{drone},k}, d_{\text{max\_range}})$$

where $d_{\text{max\_range}} = 8.0$ is the sensor's maximum detectable range. All 16 readings are normalized to $[0, 1]$ by dividing by $d_{\text{max\_range}}$ before being passed to the neural network. A normalized value of 1.0 indicates completely unobstructed free space up to the maximum range, while a value approaching 0.0 indicates an imminent physical collision.

Why LiDAR is the "Trusted" Sensor. LiDAR readings are computed directly from the ground-truth physics engine. Unlike neighbor broadcasts, which can be falsified by a compromised drone, LiDAR measurements cannot be spoofed by another agent's software. This fundamental asymmetry between trusted local sensing and untrusted peer communication is the core insight that motivates the trust evaluation module introduced in Phase C.

LiDAR Upgrade in Phase B — From 16 Rays to 192 (48 features).

The Phase A configuration of 16 individual rays is sufficient for open-arena navigation, but it becomes structurally inadequate in obstacle-rich environments. To understand why, consider

what a single ray actually encodes: the distance to the nearest surface along one specific direction. With 16 rays at 22.5° angular spacing, an obstacle that sits between two adjacent ray directions may be partially or entirely missed. More critically, a single ray cannot distinguish between a thin wall, a small isolated peg, and the curved edge of a large cylinder — all three return the same scalar distance reading. In Phase A, where the only obstructions are other drones and flat arena walls, this ambiguity is tolerable. In Phase B, where the environment contains circular obstacles of widely varying radii — from compact pebbles of radius 0.2 to large boulders of radius 2.5 — the policy needs to understand not just how far away the nearest surface is, but what kind of geometry it is approaching.

The upgraded sensor addresses this by dividing the 360° field of view into 16 sectors — maintaining the same angular coverage as Phase A — but firing 12 densely packed rays within each sector instead of one. This produces 192 total rays. However, passing all 192 raw distances directly to the neural network would be wasteful and would introduce significant redundancy, since the 12 readings within a single sector are spatially correlated. Instead, three statistical aggregates are computed per sector:

- **Minimum distance** — the closest surface detected within the sector. This preserves the collision-critical signal: if any ray within the sector hits something nearby, the minimum captures it regardless of where within the sector it falls. A single-ray design would miss this if the obstacle happened to sit between two ray directions.
- **Mean distance** — the average distance across all 12 rays in the sector. This captures the overall spatial openness in that direction. A high mean alongside a low minimum indicates a narrow protrusion in otherwise open space — the sector is broadly clear but has a spike hazard. A uniformly low mean indicates a large, space-filling obstacle. The mean is what tells the policy whether a sector is generally passable or generally blocked.
- **Standard deviation** — the spread of distances within the sector. This is the geometric complexity indicator. A near-zero standard deviation means all 12 rays return approximately the same distance — the surface in that sector is flat and uniform, consistent with a wall or a large smooth obstacle face. A high standard deviation means some rays hit close surfaces and others reach far — the sector contains complex geometry: jagged edges, gaps between obstacles, or the curved boundary of a circular obstacle where rays at different angles diverge rapidly. The standard deviation is what allows the policy to distinguish a flat wall from a curved obstacle from a cluttered gap, without the policy network needing to process all 192 raw readings.

Together, these three statistics per sector — minimum, mean, and standard deviation — form a compact geometric signature that characterizes the nature of the obstruction in each direction, not just its distance. A large boulder presents as: low minimum (close surface), low mean (fills the sector), low standard deviation (smooth curve). A narrow pillar presents as: low minimum (spike hit), high mean (space around it is open), high standard deviation (only a few rays hit it). A cluttered corridor presents as: moderate minimum, low mean, very high standard deviation (many surfaces at different depths). This 3-tuple per sector, replicated across 16 sectors, gives the policy a 48-dimensional spatial vocabulary that is both compact enough to process efficiently and rich enough to discriminate between the obstacle types the Phase B environment generates.

This is also the reason a purely descriptive single-ray design cannot be repaired simply by adding more rays: 192 raw readings would give the network more information in principle, but with no structure imposed on it, the network would need to learn the aggregation itself — a substantially harder optimization problem. Pre-computing the physically meaningful statistics makes the learning problem tractable while preserving all the geometric information that matters for navigation.

Neighbor Communication — Untrusted Channel (45 features in Phase A, 135 in Phase B). Each drone broadcasts its kinematic state to all neighbors. In Phase A, each of the $N - 1 = 9$ neighbors occupies a 5-dimensional communication slot: relative position $(p_j - p_i)/W$ normalized by arena width (2 features), normalized velocity $v_j/v_{\max}$ (2 features), and an activity flag indicating whether the neighbor remains active in the episode (1 feature). This produces $9 \times 5 = 45$ communication features. Combined with the 22 self-kinematics and LiDAR features, the total Phase A observation is 67 features.

In Phase B, the communication slot is expanded to 15 features per neighbor, adding sensor-based relative position and velocity when within sensor range; broadcast-based relative position, velocity, and stagnation indicator; visibility and communication availability flags; position and velocity discrepancy metrics that serve as early trust indicators; and a trust confidence flag encoding whether both sensor and broadcast evidence are available, sensor only, or neither. This produces $9 \times 15 = 135$ neighbor features. Combined with self-kinematics (6), LiDAR (48), and contextual features (13, including position history, congestion index, and mean discrepancy), the Phase B actor observation is approximately 202 features. A 530-dimensional global state vector is appended for the centralized critic during training, bringing the full training observation to 732 features.

3.2.2 Action Space

The action space is continuous and two-dimensional. At each timestep t, the actor network for agent i outputs:

$$a_i^t = [\Delta v_x, \Delta v_y]$$

Policy network outputs lie in the normalized range $[-1.0, 1.0]$, which are scaled by an acceleration factor ($\delta t \times k$, where $\delta t = 0.1$ and $k = 5.0$ in Phase A, $k = 10.0$ in Phase B) to update velocity:

$$v_i^{t+1} = v_i^t + a_i^t \times \delta t \times k$$

The resulting velocity is magnitude-clamped to $v_{\max} = 2.0$ m/s. In Phase B, the effective maximum velocity is further dynamically reduced based on local congestion:

$$v_{\max}^{\text{local}} = \max(v_{\max} \times e^{-0.15 \times n_c}, 0.75)$$

where n_c is the number of neighbors within 1.0 unit distance. This adaptive speed limiting prevents high-velocity collisions in dense clusters.

Continuous control is essential here because drone swarm navigation requires smooth trajectory planning, fine-grained steering, and rapid adaptation to local geometry. Discrete action spaces cannot produce the nuanced collision avoidance behavior required in dense multi-agent settings, nor do they model realistic quadrotor dynamics accurately.

3.3 Reward Function Design

In decentralized swarm navigation, the reward function carries the entire burden of behavioral specification. Without a centralized controller, a single scalar signal must simultaneously encode goal-directed progress, safe inter-agent spacing, kinematic regulation, and failure avoidance. The following categories define the minimal set of reward signals required to stabilize cooperative MARL training.

3.3.1 Core Reward Categories

1. **Goal-Seeking Gradient.** Sparse rewards (e.g., +1 only at the goal) are unlearnable when ten randomly initialized agents must discover the target through undirected exploration. A potential-based goal field provides a dense per-step signal proportional to the change in distance to the goal, giving PPO a usable gradient from the first episode. This is paired with an existential penalty — a small negative reward every timestep — to make idleness strictly suboptimal and force time-efficient trajectories.
2. **Swarm Topology: Cohesion vs. Repulsion.** Cohesion keeps agents within mutual sensing range, enabling collective behavior. Repulsion prevents cohesion from collapsing the swarm into a physically overlapping mass as all ten agents funnel toward the same goal. These two forces operate at different spatial scales — cohesion is rewarded at medium range while repulsion is penalized at close range — creating a stable equilibrium band where agents maintain safe spacing without drifting apart.
3. **Kinematic Safety.** In continuous physics, drones carry momentum and cannot stop instantaneously. Pure distance-based penalties are insufficient: an agent at 2.0 m/s toward a neighbor 0.5 m away has only two timesteps ($\delta t = 0.1$) to react. Kinematic rewards penalize dangerous dynamic states — high speeds in dense clusters, and trajectories converging toward a collision — transforming the safety signal from reactive to predictive.
4. **Terminal Conditions.** Catastrophic failures (wall contacts, inter-agent collisions) terminate the episode with large negative penalties, ensuring the network treats them as hard constraints. Goal arrival provides a large positive bonus that dominates all other signals, maintaining goal-directed motivation over purely defensive behavior.

3.3.2 Phase A: Foundational Coordination

In Phase A, the environment contains no static obstacles and no adversarial agents. The reward function focuses strictly on establishing reliable goal-directed navigation and collision-free swarm cohesion as the behavioral foundation for all subsequent phases.

Potential Goal Field.

$$R_{\text{goal}} = 10.0 \times [0.995 \times (-d_{\text{now}}) - (-d_{\text{old}})]$$

The primary driving force of the entire reward structure. The term $(d_{\text{old}} - d_{\text{now}})$ measures how much closer the agent moved toward the goal in the current timestep. The 0.995 discount introduces a subtle time pressure, encouraging faster convergence without producing instability early in training. The multiplier of 10.0 ensures goal progress dominates the reward landscape over all other dense components.

Component	Formula	Purpose
Goal Field	$10.0 \times [0.995 \times (-d_{\text{now}}) - (-d_{\text{old}})]$	Dense gradient guiding agents toward the goal
Existential Penalty	-0.05 per step	Enforces time-efficient trajectories
Cohesion Reward	$+0.01$ per neighbor in $[0.6\text{ m}, 4.0\text{ m}]$	Maintains group cohesion without rewarding dangerous proximity
CoM Expansion	$\text{clip}(30.0 \times \Delta d_{\text{CoM}}, -3.0, 3.0)$	Proactive anti-clumping before collision threshold
School-Zone Speed Limit	$-((v - v_{\text{safe}})/v_{\max})^2 \times n_{\text{close}} \times 5.0$	Reduces kinetic energy in dense clusters
Social Distancing	$-(0.4 - d_{\text{neighbor}}) \times 50.0$	Soft-wall repulsion above the collision boundary
Success Bonus	$100.0 + 50.0/(1.0 + v)$	Rewards arrival with smooth deceleration incentive
Wall Collision	-100.0 (terminal)	Severe penalty for boundary contact
Drone Collision	-50.0 (terminal)	Penalty for inter-agent physical overlap

Existential Penalty.

$$R_{\text{time}} = -0.05 \text{ per timestep}$$

Applied unconditionally at every step, this penalty makes any trajectory that does not actively progress toward the goal strictly suboptimal over time, preventing the policy from learning to loiter safely rather than navigate efficiently.

Cohesion Reward.

$$R_{\text{cohesion}} = +0.01 \text{ per neighbor within } [0.6\text{ m}, 4.0\text{ m}]$$

Maintains operational proximity between agents without rewarding dangerous clustering. The lower bound of 0.6 m ensures cohesion is never active at close range. The coefficient of 0.01 is intentionally small — cohesion must shape spatial configuration as a secondary consideration, not compete with goal-directed motion.

Center-of-Mass Expansion.

$$R_{\text{expansion}} = \text{clip}(30.0 \times (d^t_{\text{CoM}} - d^{t-1}_{\text{CoM}}), -3.0, 3.0)$$

When neighbors are detected within 0.55 m, the agent is rewarded for moving away from the local center of mass of those neighbors. This is the primary anti-clumping mechanism, creating proactive outward pressure before the collision threshold is reached. The multiplier of 30.0 compensates for the fact that raw CoM displacement per timestep is typically below 0.05 m and would otherwise be numerically negligible. The clip bounds prevent reward spikes that

would destabilize critic value estimation.

School-Zone Speed Limit.

$$R_{\text{speed}} = -((v - v_{\text{safe}})/v_{\max})^2 \times n_{\text{close}} \times 5.0$$

When close neighbors are within 0.55 m, this penalizes speeds above $v_{\text{safe}} = 0.35 \times v_{\max}$. The quadratic form matches the physics — collision severity scales as v^2 — and the multiplication by n_{close} makes the penalty context-aware: strict in crowded regions, inactive in open space.

Social Distancing Penalty.

$$R_{\text{social}} = -(0.4 - d_{\text{neighbor}}) \times 50.0$$

Activates when any neighbor enters the danger zone below 0.4 m — just above the physical collision threshold of 0.25 m. The 0.15 m warning band between these two thresholds gives the agent approximately 1–2 timesteps of reaction time to initiate evasive action before the episode terminates.

Success Bonus.

$$R_{\text{success}} = 100.0 + 50.0/(1.0 + v)$$

The fixed component of 100.0 ensures goal arrival is always the globally preferred outcome. The velocity-dependent term discourages high-speed overshoots: at $v = 0$ the bonus reaches +50.0, at $v = 2.0$ m/s it reduces to approximately +16.7, creating a meaningful 33-point incentive for controlled deceleration on arrival.

Collision Penalties.

Wall: -100.0. Drone-on-drone: -50.0. Both terminal.

Wall collision is penalized twice as severely because arena boundaries are static and fully observable — a wall collision represents a more fundamental policy failure than a drone collision, which can occur even in geometrically constrained situations. The asymmetry also encodes a practical preference: if trapped between a neighbor and the boundary, the policy is incentivized to accept the smaller drone-collision penalty rather than fly into the wall.

3.3.3 Phase B: Environmental Complexity

Phase B introduces static circular obstacles and denser configurations. The reward function shifts from purely proximity-based signals to dynamic kinematic metrics, since in obstacle-rich environments distance alone is no longer a sufficient safety indicator.

Dense Progress.

$$R_{\text{goal}} = 100.0 \times (d_{\text{old}} - d_{\text{now}}) - 0.25$$

The goal multiplier is scaled up to 100.0 because obstacle navigation frequently requires lateral detours that temporarily increase distance to the goal. A stronger signal maintains goal-directed motivation through these necessary diversions. The per-step penalty of -0.25 replaces the Phase A existential penalty, scaled proportionally to the longer episode limit of 800 steps.

Component	Formula	Purpose
Dense Progress	$100.0 \times (d_{\text{old}} - d_{\text{now}}) - 0.25$	Strong goal gradient robust to obstacle detours
Stagnation Penalty	$-\min((n_{\text{stagnant}} - 50) \times 0.25, 25.0)$	Penalizes deadlocks and local minima traps
TTC Penalty	$-10.0 \times (1.0 - \text{TTC})^2$	Predictive safety signal based on closing speed
Traffic Jam Dispersion	$-50.0 \times \sigma(-10.0(d - 0.5))$	Forces rerouting around stalled neighbors
Action Smoothness	$-0.05 \times \lvert a_t - a_{t-1} \rvert^2$	Smooth control transitions and stable dynamics
Velocity Alignment	$+0.2 \times \cos(\theta_{ij}) \times \max(0, \hat{u}_g \cdot \hat{v}_i)$	Rewards coherent swarm flow toward goal
LiDAR Clearance	$+0.2 \times$ (Forward_Mean/8.0)	Incentivizes navigating toward open space
Terminal Penalties	-500.0 (wall, obstacle, drone)	Hard constraint ensuring survival in dense environments

Stagnation Penalty.

$$R_{\text{stagnation}} = -\min((n_{\text{stagnant}} - 50) \times 0.25, 25.0)$$

If no measurable progress is made for more than 50 consecutive steps, an escalating penalty begins accumulating. The 50-step grace period allows legitimate short-term detours without triggering the penalty. Beyond that, the linear growth with a cap of -25.0 discourages agents from hovering in dead-ends rather than backtracking — a failure mode that arises specifically in obstacle environments where backtracking produces temporary goal-field losses.

Time-to-Collision Penalty.

$$R_{\text{TTC}} = -10.0 \times (1.0 - \text{TTC})^2$$

Penalizes agents whose current velocity vector will produce a collision within 1.0 second, computed from the closing speed between agents. The key advantage over distance-based penalties is context-sensitivity: two agents slowly passing each other at 0.3 m with diverging velocities receive no penalty, while two agents at 1.5 m converging at high speed receive a strong one. This distinction is essential in obstacle environments where controlled close-proximity navigation through narrow gaps must not be penalized.

Traffic Jam Dispersion.

$$R_{\text{dispersion}} = -50.0 \times \sigma(-10.0 \times (d - 0.5))$$

In narrow corridors, a stalled lead drone causes trailing agents to compress behind it, producing chain-reaction collisions when it begins moving again. The sigmoid penalty forces trailing

agents to route around stalled neighbors rather than queue directly behind them. The sigmoid is preferred over a linear penalty because the dispersion behavior needs to activate sharply below a specific distance threshold while remaining negligible at larger separations.

Action Smoothness Penalty.

$$R_{\text{smooth}} = -0.05 \times |a_t - a_{t-1}|^2$$

Penalizes abrupt action changes between consecutive timesteps. Beyond physical realism — real quadrotors cannot sustain large instantaneous velocity changes — smooth action sequences produce smoother value function landscapes that reduce variance in the critic's gradient estimates.

Velocity Alignment Bonus.

$$R_{\text{align}} = +0.2 \times \cos(\theta_{ij}) \times \max(0, \hat{u}_g \cdot \hat{v}_i)$$

Rewards directional agreement with nearby neighbors, gated by whether the agent itself is moving toward the goal. The gate prevents agents from learning to align with neighbors that are moving away from the goal. The combined effect is coherent swarm flow through obstacle fields, reducing the crossing trajectories that produce lateral collisions.

LiDAR Clearance Reward.

$$R_{\text{clearance}} = +0.2 \times (\text{Forward\_Mean}/8.0)$$

A small continuous reward proportional to average forward-facing LiDAR readings, normalized by the maximum sensor range. This creates a soft preference for clear paths over obstacle-adjacent ones, causing agents to naturally orient toward open corridors when multiple route options are available.

Terminal Penalties.

All collision types: -500.0 (terminal).

The tenfold increase from Phase A reflects the substantially greater density of collision opportunities in obstacle-rich environments. At Phase A magnitudes, the optimizer may accept occasional collisions as a tolerable tradeoff for faster goal convergence through tight corridors. The -500.0 penalty makes this tradeoff strictly unacceptable, enforcing survival as a hard constraint.

3.4 CTDE Architecture

3.4.1 Centralized Critic

The critic receives the full global swarm state during training. Each drone contributes:

- Position
- Velocity
- LiDAR sweep

For a 10-drone swarm:

$$10 \times 52 = 520$$

global critic features are used.

3.4.2 Decentralized Actor

The actor uses only local observations:

- Self-kinematics
- LiDAR readings
- Neighbor communication
- Temporal context
- Trust anomaly indicators

The actor observation space contains approximately 130 features.

3.5 Simulation Framework

The simulation environment is implemented using:

- PettingZoo ParallelEnv API
- Stable-Baselines3 PPO
- PyTorch neural networks
- NumPy
- Matplotlib
- PyGame rendering engine

The environment supports continuous drone dynamics, LiDAR sensing, collision handling, obstacle navigation, and dense swarm evaluation.

4 Implementation Progress

4.1 Modules Developed

4.1.1 Swarm Environment

A complete continuous-space drone swarm environment has been implemented with:

- Continuous velocity control
- Physics-based movement
- Collision handling
- Goal navigation
- Multi-agent synchronization

4.1.2 LiDAR System

The environment currently supports:

- 16-ray LiDAR sensing
- Wall detection
- Drone proximity sensing
- Sensor normalization

4.1.3 Reward Engineering

Several reward functions have been implemented:

- Goal-directed reward
- Cohesion reward
- Cluster-expansion reward

- Social-distancing penalty
- Speed-limit regulation
- Collision penalties

4.1.4 Evaluation Framework

The evaluation pipeline supports:

- Random scenario generation
- Clustered stress testing
- Edge-case testing
- Statistical reproducibility

4.2 Code Architecture

The codebase is modularized into:

- Environment layer
- PPO training layer
- Evaluation layer
- Visualization layer

This modular design improves maintainability and future adversarial extensions.

4.3 Integration Status

Phase	Status
Phase A - Basic Swarm Coordination	Completed
Phase B - Obstacle Navigation	In Progress
Phase C - Trust Evaluation	Pending
Phase D - Full Adversarial Defense	Pending

Table 1: Current Integration Status

4.4 Current Completion Level

The project is currently estimated to be approximately 60% complete.

Completed components:

- Environment implementation
- PPO integration
- Reward engineering
- Evaluation framework

Pending components:

- Trust gating integration
- Adversarial communication
- Full TA-MAPPO evaluation

5 Simulation Setup

5.1 Simulation Environment

Parameter	Value
Arena Size	20×20
Number of Drones	10
Time Step	0.1 s
Maximum Episode Length	600 steps
Maximum Velocity	2.0
LiDAR Rays	16
Action Space	Continuous (v_x, v_y)

Table 2: Simulation Parameters

5.2 Training Parameters

Hyperparameter	Value
RL Algorithm	PPO
Learning Rate	5×10^{-5}
Entropy Coefficient	0.005 – 0.03
Rollout Workers	10
Reward Normalization	Enabled
Observation Normalization	Disabled

Table 3: Training Hyperparameters

5.3 Test Scenarios

The environment supports:

- Random spawn scenarios
- Dense cluster configurations
- Obstacle navigation maps
- Future adversarial communication settings

5.4 Performance Metrics

The following metrics are used:

- Success Rate
- Collision Rate
- Time to Goal
- Convergence Stability

6 Results Obtained So Far

6.1 Preliminary Simulation Results

Scenario	Success Rate	Standard Deviation
Random Spawn	99.68%	±0.19
Dense Cluster	95.78%	±0.42

Table 4: Phase A Evaluation Results

6.2 Comparative Analysis

Traditional Vanilla MAPPO systems suffer from:

- Dense-cluster instability
- Swarm panic behavior
- Collision cascades

The proposed framework significantly improves coordination performance in dense swarm conditions. The dense-cluster success rate improved from:

$$89.45\% \rightarrow 95.78\%$$

6.3 Interpretation of Results

The obtained results demonstrate:

- Stable decentralized coordination
- Improved congestion handling
- Effective collision avoidance
- Reliable continuous-control navigation

These findings validate the feasibility of extending the framework toward adversarial resilience.

7 Novelty and Research Contribution

7.1 Novelty of the Proposed Work

The novelty of this work lies in integrating:

- Bio-inspired trust evaluation
- Trust-aware communication filtering
- Multi-Agent Reinforcement Learning
- Continuous decentralized swarm control

within a unified framework.

7.2 Improvements Over Existing Methods

Existing MARL systems:

- Assume reliable communication
- React only after failures occur
- Lack proactive trust evaluation

The proposed TA-MAPPO framework suppresses deceptive information before policy inference

and improves dense-swarm resilience.

7.3 Contribution to the Field

Potential contributions include:

- Byzantine-resilient swarm intelligence
- Adversarially robust MARL systems
- Bio-inspired trust-aware robotics
- Reproducible adversarial swarm simulators

8 Publication Readiness

8.1 Target Journals and Conferences

Potential publication venues include:

- IEEE Transactions on Intelligent Transportation Systems
- Engineering Applications of Artificial Intelligence
- IEEE ICRA Workshops
- AAMAS MARL Workshops

8.2 Tentative Paper Title

TA-MAPPO: Trust-Aware Multi-Agent Reinforcement Learning for Resilient Drone Swarms

8.3 Proposed Paper Outline

1. Introduction
2. Related Work
3. TA-MAPPO Framework
4. Trust-Gated Architecture
5. Experimental Setup
6. Results and Analysis
7. Ablation Studies
8. Conclusion and Future Work

8.4 Additional Experiments Required

Before publication-quality submission, the following experiments are required:

- Full adversarial Phase C implementation
- Trust-score convergence analysis
- ROC curve evaluation
- Confusion matrix analysis
- Baseline comparisons
- Scalability testing

9 Challenges and Future Work

9.1 Current Limitations

Current limitations include:

- No active traitor implementation

- Only 2D simulation support
- Static obstacle assumptions
- Limited scalability evaluation

9.2 Technical Challenges Faced

Major technical challenges include:

- Dense swarm stability
- Reward balancing
- PPO convergence stability
- Environment solvability

9.3 Planned Improvements

9.3.1 Near-Term Improvements

- Implement deceptive communication
- Add trust-score heatmaps
- Improve obstacle navigation

9.3.2 Mid-Term Improvements

- Full TA-MAPPO adversarial evaluation
- Baseline comparisons
- Hyperparameter optimization

9.3.3 Long-Term Improvements

- Transition to realistic 3D environments
- PX4 integration
- Hardware-in-the-loop testing

9.4 Projected Timeline

Timeline	Planned Work
Next 2 Weeks	Complete Phase B
Next 1 Month	Implement Phase C
Next 2 Months	Full TA-MAPPO Evaluation
Final Stage	Publication Experiments

Table 5: Projected Timeline

10 Conclusion

This project establishes a strong foundation toward resilient decentralized swarm navigation under adversarial communication conditions. The completed implementation demonstrates highly stable continuous-control swarm coordination and effective congestion handling in dense multi-agent environments.

The proposed TA-MAPPO framework combines:

- Multi-Agent Reinforcement Learning
- Bio-inspired trust mechanisms
- Trust-aware communication filtering
- Continuous decentralized control

Future work will focus on adversarial communication modeling, trust-aware defensive evaluation, large-scale swarm testing, and realistic hardware deployment.